# **InfectPrompt**: Inducing Vulnerabilities in C/C++ Functions Using LLMs for GNN-based Vulnerability Detection

Joelle Dizon*, Lauren Kwon†, Caden Hicks‡, Ekincan Ufuktepe‡, Prasad Calyam‡, Kannappan Palaniappan‡
*Dept. of Computer Science, Lehigh University, Bethlehem, USA
Email: jkd227@lehigh.edu
†Dept. of Computer Science, Iowa State University, Ames, USA
Email: isoo0301@iastate.edu
‡Dept. of Electrical Engineering and Computer Science, University of Missouri, Columbia, USA
Emails: {cjhbv4, euh46, calyamp, pal}@missouri.edu

***Abstract*—Timely detection of vulnerabilities in C/C++ code is essential for software security, yet machine learning models struggle due to small, imbalanced datasets. We present *InfectPrompt*, a framework that uses Large Language Models to inject realistic vulnerabilities into safe code, creating a semi-synthetic dataset called *ResourceVul*. By augmenting real-world data with LLM-generated samples, we trained a Graph Neural Network and observed consistent improvements in accuracy (+1.3 percentage points) and F1-score (+2.5 points) compared to training on real data alone. Our analysis shows that LLMs are especially effective at generating certain memory safety vulnerabilities, though they cannot fully replace real-world samples. This work presents a scalable approach to enriching vulnerability datasets and highlights the potential of combining synthetic and real-world examples to enhance automated vulnerability detection.**

***Index Terms*—vulnerability detection, data augmentation, data generation, software security, large language models**

## I. Introduction

Software systems face significant security challenges as codebases expand in size and complexity, increasing the risk of vulnerabilities that can result in unauthorized access, data breaches, and service interruptions. These risks are compounded by the accelerating pace of software development and code reuse in large-scale, often open-source, systems. To mitigate this, machine learning-based vulnerability detection, particularly deep learning models trained on source code, has emerged as a promising approach due to its ability to automatically learn complex syntactic and semantic patterns that correlate with known security flaws [1]. However, the effectiveness of these models is fundamentally constrained by the quality, quantity, and representativeness of training data. Many existing datasets are too small [2], imbalanced [3], or lack the fine-grained annotations [4] necessary for effective learning. These limitations hinder generalization to real-world codebases and reduce the reliability of such models in practical deployment.

To address this gap, recent work explored using Large Language Models (LLMs) to generate synthetic code snippets exhibiting known vulnerabilities, guided by Common Weakness Enumerations (CWEs) and Common Attack Pattern Enumerations and Classifications (CAPEC) [5]. 615 CAPEC-aligned samples across Java, Python, and JavaScript using models such as GPT-4o, LLaMa, and Claude were generated. Manual review of 30 randomly selected samples per language found that 92% accurately reflected the intended vulnerabilities. While promising, the exclusive reliance on synthetic code limits realism. These examples fail to capture the complexity, coding diversity, and subtle patterns found in real-world vulnerabilities.

We propose a new semi-synthetic dataset constructed using DeepSeek-Coder-1.3b-instruct. Our approach focuses on injecting five CWEs, frequently observed in the PrimeVul [6] and DiverseVul [3] datasets, into safe, real-world C/C++ functions from PrimeVul. Vulnerabilities are introduced using carefully engineered prompts designed to preserve the original code's syntax, structure, and semantics. The resulting dataset contains both the original non-vulnerable functions and their vulnerability-injected counterparts, offering a more realistic and fine-grained resource for training and evaluating automated vulnerability detection systems at scale.

This study examines the effectiveness of using LLM-injected vulnerabilities to improve real-world vulnerability detection. We evaluated the effectiveness of prompt-based CWE injection for training and testing GNNs on graph-level classification tasks. Our experiments compare three training settings: (1) using only

real data, consisting of non-vulnerable and vulnerable functions; (2) combining real data with 50% of LLM-injected vulnerable functions; and (3) combining real data with 100% of LLM-injected vulnerable functions. All models are evaluated on a held-out test set, including naturally occurring, real-world vulnerable and non-vulnerable functions. Table II presents the detailed data composition in all training/testing configurations, including the distributions of injected CWE types and real samples in each setting. We aim to answer the following research questions.

**RQ1: Does training on LLM-injected vulnerabilities improve generalization to real-world code?** We compare models trained on real, non-vulnerable functions with and without synthetic vulnerabilities to assess whether prompt-injected CWE examples help detect vulnerabilities in previously unseen, natural code.

**RQ2: Can LLM-injected vulnerabilities serve as a substitute for real-world vulnerable samples in training?** We evaluate whether training solely on LLM-injected vulnerabilities can match the performance of models trained on real vulnerable samples, and whether a hybrid approach (real+synthetic) offers additional benefits.

**RQ3: How does the success rate of LLM-injected vulnerabilities vary across different CWE types, and what does this imply about the suitability of synthetic data augmentation for each vulnerability class?** We investigate how the success rate of LLM-injected vulnerabilities varies across different CWE types, analyzing the retention of usable vulnerable samples generated for each class. This research question aims to uncover which vulnerability classes are synthesized more effectively by the LLM, providing insight into the uneven distribution of synthetic data quality and its implications for training data augmentation.

This paper makes the following key contributions:

- We propose a novel method to generate semi-synthetic vulnerable code by injecting CWE-specific flaws into real-world C/C++ functions using LLM. We publicly share our code for the vulnerable code generation process[1].
- Our dataset pairs original safe functions with their vulnerability-injected counterparts, enabling fine-grained training and evaluation.
- We systematically evaluate how augmenting training data with LLM-injected vulnerabilities improves Graph Neural Network (GNN) performance on real-world benchmarks.
- We provide a detailed analysis of which vulnerability types benefit the most from synthetic data augmentation.
- We investigated the feasibility of using synthetic vulnerabilities as a substitute for real vulnerable samples in training.

The remainder of this paper is organized as follows. Section II discusses the limitations of existing vulnerability detection models and datasets, motivating the need for this study. Section III, introduces the overall framework of *InfectPrompt* and outlines the training process of the GNN model. Section IV, presents the architectural details of the GNN model. In section V, we present the findings of our study, address the research questions, and discuss the limitations of our approach, including potential threats to validity. Finally, section VI summarizes our contributions and outlines the directions for future work.

## II. Related Work

### A. Limits of Existing Vulnerability Detection Models

Existing vulnerability detection models, including static analysis and machine learning (ML) based techniques, exhibit fundamental limitations that hinder their effectiveness in real-world settings. Open-source static analysis tools such as Flawfinder and RATS have less-than-ideal True Positive Rates (TPRs) of 33% and 54%, respectively [7]. They detect less than half of the vulnerabilities in a labeled dataset. Furthermore, a commercial tool such as CheckMarx has a TPR of 31.9%, meaning that it can detect only about 31.9% of vulnerabilities in a labeled data set. These results are reasonable, given that these tools rely on expert-defined rules and patterns. Due to the diverse nature of vulnerabilities, it is not feasible to capture all relevant patterns, leading to limited detection performance.

ML-based approaches, while showing promise, are particularly susceptible to overfitting when trained on datasets that are small, imbalanced, or lack structural and semantic diversity. Models trained and tested on disjoint subsets of the DiverseVul [3] dataset showed low F1 scores, the highest-scoring model being NatGen at 47.15%. Furthermore, following the introduction of stricter labeling and evaluation methods in the PrimeVul [6] benchmark, models trained on popular datasets like BigVul [4] showed degraded performance when evaluated on PrimeVul. The highest scoring model for that experiment was CodeT5, with a low F1 score of 5.82%. These statistics suggest a lack of generalization; rather than learning meaningful vulnerability patterns, models may instead rely on irrelevant correlations such as syntactic features, resulting in a significant drop in performance when applied to unseen data.

### B. Limitations of Existing Vulnerability Datasets

DiverseVul [3] is a dataset that extracts C/C++ code files from vulnerability fixing commits identified on

[1]https://github.com/sqam-lab/ResourceVul

two major security issue platforms. It contains 18,945 vulnerable functions spanning 150 CWE types, along with 330,492 non-vulnerable functions. However, DiverseVul exhibits several critical limitations. First, it suffers from high label noise as it identifies vulnerability based on commit changes. This approach incorrectly labels non-vulnerable function modifications and whitespace-only changes as vulnerability fixes. Manual analysis on randomly selected 50 samples reveals that only about 60% of the vulnerable function labels are accurate. Second, the dataset is highly imbalanced in terms of class distribution, with a significantly larger number of non-vulnerable functions compared to vulnerable functions. This imbalance can bias the model toward predicting the majority class (non-vulnerable). Third, the distribution of fixing commits across CWE types is uneven, with some categories having far more exponents than others. As a result, models trained on DiverseVul are disproportionately exposed to certain vulnerability types, which can hinder their ability to generalize to underrepresented or rare CWEs. These issues collectively impact model robustness and contribute to degraded performance, particularly when evaluated on unseen projects or less frequent vulnerability categories.

PrimeVul [6] is a comprehensive vulnerability dataset that integrates security-related commits and modified functions from several prominent datasets, including BigVul [4], CrossVul [8], CVEfixes [9], and DiverseVul [3]. To ensure data quality, PrimeVul applies a rigorous de-duplication process through normalization and MD5 hashing, eliminating not only exact duplicates but also functions that differ only in formatting. The dataset is partitioned into training, validation, and test sets in chronological order to prevent data leakage and better reflect real-world deployment scenarios. Despite these enhancements, PrimeVul exhibits several limitations. The dataset is highly imbalanced, comprising 6,968 vulnerable and 228,800 benign functions. This significant class imbalance challenges the training and evaluation of models, particularly when relying on metrics like accuracy or F1-score, which can be misleading in skewed distributions. Furthermore, the limited contextual information within the isolated functions and potential labeling inaccuracies add to the difficulty of the dataset. These challenges can compromise model effectiveness and reduce the applicability to more complex, real-world software environments.

Furthermore, SecVulEval [10] was introduced as a benchmarking framework for evaluating vulnerability detection across multiple datasets and models. While SecVulEval provides a valuable unified benchmark, its integration into our study was not feasible due to time constraints and limited overlap with our selected CWE categories. We highlight it as an important direction for future work to situate InfectPrompt within broader evaluation pipelines.

## III. Methodology

We utilize the complete set of non-vulnerable functions from the *PrimeVul* dataset as our foundation. Through cross-dataset analysis of *PrimeVul* and *DiverseVul*, we identify the five most prevalent CWE types common to both repositories. Selecting CWEs shared across both datasets ensures that our injection targets represent well-established and broadly relevant vulnerability categories, thereby enhancing the practical significance and generalizability of the synthetic vulnerabilities generated. These CWEs serve as injection targets for generating synthetic vulnerable code.

### A. InfectPrompt Framework

In Fig. 1, we present the proposed framework, which consists of three primary components: **(1) a targeted natural language prompt, (2) a CWE identifier with a brief description of the CWE**, and **(3) a non-vulnerable function from PrimeVul**. These are provided as input to an LLM. The prompt contextualizes the vulnerability within a specific software security concern, while the CWE identifier and its description guide the LLM's attention toward a known vulnerability category. The LLM then processes the non-vulnerable function alongside these components to generate a corresponding vulnerable version aligned with the specified CWE. To ensure broad coverage and class balance, we randomly divided the 218,525 non-vulnerable functions from PrimeVul into five disjoint subsets, each associated with one of the five most frequent CWEs: CWE-787 (Out-of-Bounds Write), CWE-125 (Out-of-Bounds Read), CWE-119 (Buffer Overflow), CWE-20 (Improper Input Validation), and CWE-416 (Use After Free). Each function in a subset was paired with a prompt that targets its assigned CWE, resulting in five distinct sets of vulnerable samples, each corresponding to a single vulnerability type. Although our current implementation focuses on the five most prevalent CWEs, this structure supports extensive customization, allowing future inclusion of a broader range of CWE types. Additionally, it facilitates control over class distribution, enabling dataset balancing between vulnerable and non-vulnerable functions, as well as supporting balanced representation across CWE types in future expansions.

We initially aimed to generate vulnerability-induced counterparts for all 218,529 non-vulnerable C/C++ functions in the dataset. To balance computational constraints with the need for high-quality data, we implemented a scalable sampling strategy aimed at preserving representative coverage across CWE types. Specifically, we targeted 4,300 validated samples per CWE type, totaling

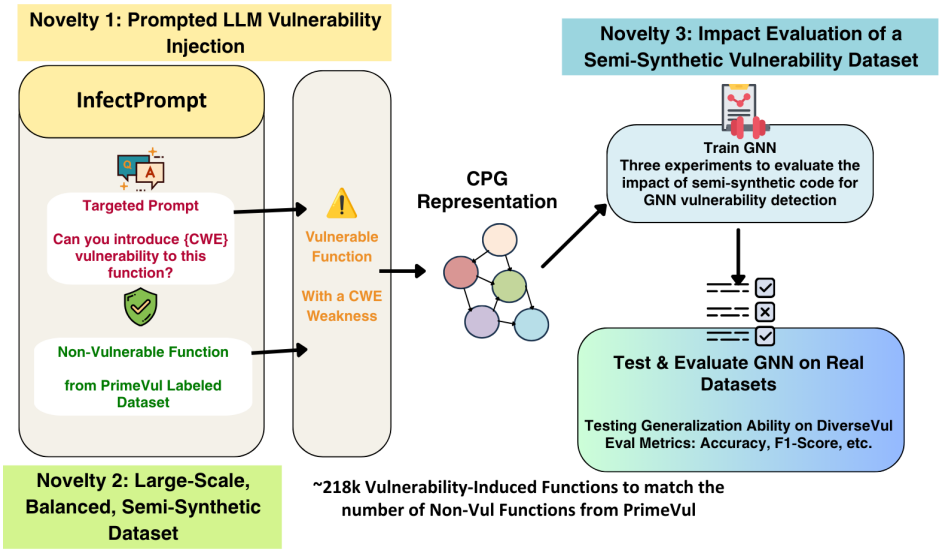



Fig. 1. Pipeline for the INFECTPROMPT vulnerability-inducing framework.

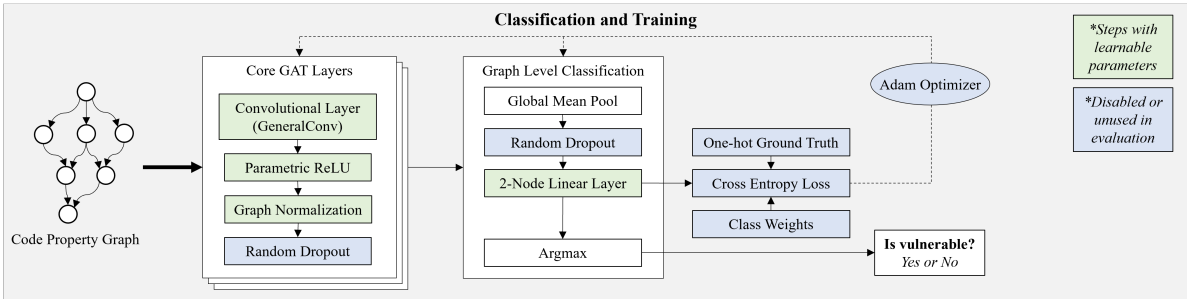



Fig. 2. Classification pipeline for the GNN code vulnerability detection system.

21,500 samples, which represent approximately one-tenth of the original dataset, to serve as a robust benchmark for vulnerability detection. To account for expected losses during filtering, we first generated approximately 6,500 synthetic vulnerable functions per CWE, resulting in a total of 32,652 samples prior to filtering. Through a rigorous filtering pipeline, we refined this to a final dataset of 19,784 validated vulnerability-injected functions. This approach ensures class balance and enhances data quality, significantly alleviating the scarcity of well-labeled vulnerable samples, as the original datasets contained fewer than 500 real-world vulnerable functions per CWE type.

We then partitioned the 182,559 non-vulnerable functions into 80% for training and 20% for testing to support the development of the GNN. Similarly, the 19,784 vulnerability-injected functions were split using the same 80/20 ratio, resulting in 15,828 samples for training and 3,956 for testing. All functions, both the 182,559 non-vulnerable and 19,784 vulnerability-injected training samples, were converted into Code Property Graphs (CPGs) and exported as DOT files for model training. We then trained the GNN model on these graph representations and conducted a comparative study to evaluate the impact of synthetic data on model performance. We designed three training configurations: We consider a base set of 159,856 non-vulnerable and 4,555 vulnerable real-world functions. The synthetic augmentation varies as follows:

1) **0% synthetic**: base only.
2) **50% synthetic**: base + 7,914 synthetic vulns.
3) **100% synthetic**: base + 15,828 synthetic vulns.

### *B. LLM Selection for Vulnerability Injection*

We selected *DeepSeek-Coder-1.3b-Instruct*[2] as the generative model for our vulnerability injection pipeline based on its alignment with several key research requirements. First, it is open source and freely available, distributed under a permissive license that supports modification and redistribution. This accessibility allows for local deployment, which is essential for ensuring

[2]https://huggingface.co/deepseek-ai/deepseek-coder-1.3b-instruct

reproducibility and maintaining control over the generation process, and significantly reducing data generation time. Unlike cloud-based inference via services like OpenRouter, which require repeated requests and introduce latency, local execution enables faster and uninterrupted batch processing of vulnerability-injected samples. Second, the model performs strongly in code synthesis and comprehension tasks, particularly when prompted with domain-specific instructions. Trained in a wide range of programming languages and software dependencies, *DeepSeek-Coder-1.3b-Instruct* demonstrates a deep understanding of code semantics and common vulnerability patterns, making it effective in transforming secure functions into their vulnerable counterparts. Furthermore, using the instruction-tuned version enhances the model's ability to interpret and follow complex natural language prompts, including those referencing specific CWE identifiers. This instruction-following capability provides fine-grained control over the injection process, enabling the generation of vulnerable code that is both syntactically plausible and semantically faithful to the original, non-vulnerable function.

### C. Prompt Design for LLM

To generate vulnerable variants of safe functions, we designed a standardized prompt template that instructed the LLM to inject a vulnerability of a specific CWE type into a given non-vulnerable function. Each prompt included (i) the CWE identifier and description and (ii) the original function to be modified. To ensure consistency, we required that the model: (1) directly alter the given function rather than producing new or unrelated ones, (2) insert exactly one inline comment to mark the injected vulnerability, and (3) avoid natural language explanations outside of the code. These design constraints are consistent with prior work emphasizing the need for prompt structure to maintain syntactic fidelity in code generation [11], [12], and they also made it easier to trace which parts of the output were altered.

Early experiments revealed several recurring issues, including truncated functions, unchanged outputs, and the generation of multiple functions in one response. To address these problems, we varied decoding hyperparameters such as temperature and top-p. Consistent with observations from other studies on LLM code generation [13], we found that higher top-p values improved diversity without substantially increasing invalid outputs. Specifically, increasing *top-p* from 0.6 to 0.95 improved structural variation while preserving validity, whereas changes in temperature often introduced instability. The final configuration (*temperature* = 0.3, *top-p* = 0.95) offered the best balance and was adopted for all large-scale vulnerability injection runs.

### D. Dataset Selection for Vulnerability Injection

We selected the *PrimeVul* dataset for augmentation due to its suitability for vulnerability injection. PrimeVul is derived from open-source software projects from the real world, ensuring that the code samples reflect practical coding patterns and development contexts. The dataset provides annotations at the function level, allowing for precise injection of vulnerabilities into a standalone code unit. This aligns well with modern ML-based vulnerability detection approaches that operate within the scope of the function. The dataset comprises a diverse set of functions from various software domains, promoting generalization across different types of projects. Moreover, each function is normalized and well-structured, reducing the need for extensive preprocessing and making it readily compatible with automated augmentation pipelines.

To inform our selection of injection targets, we additionally analyzed the *DiverseVul* dataset. Although DiverseVul was not directly used in training or evaluation, its inclusion in our analysis provided a broader perspective on the types of vulnerabilities that commonly occur across datasets. This allowed us to identify CWE categories that are both prevalent and consistently labeled, guiding our choice of high-impact, broadly applicable injection targets.

### E. Filtering LLM-generated Code

Despite careful prompt engineering, many LLM-generated outputs failed to meet the requirements for CPG conversion and vulnerability analysis. We observed several recurring issues:

1) Truncated or syntactically invalid code.
2) Outputs containing multiple functions, confusing the range of vulnerability.
3) Unchanged and superficially modified functions (e.g., adding descriptions), risking mislabeling.

To address these issues, we implemented a multistage filtering and validation pipeline. First, we automatically removed files containing multiple functions, truncated outputs, or syntactically invalid code. Second, we normalized both original and generated functions and discarded unchanged outputs to ensure meaningful transformations. Finally, we performed manual spot checks on randomly sampled functions to confirm that injected flaws reflected the intended vulnerabilities. This combination of automated and manual validation provided both scalability and correctness.

The second stage focused on removing the syntactically incomplete or malformed code. We implemented a Python script that recursively scans each generated file and verifies that all opening and closing braces are correctly balanced. Files that did not pass this check were considered truncated or invalid and were excluded.

In the final stage, we wrote a Python script to normalize both the original and generated functions by removing comments, flattening multiline function headers, stripping redundant whitespace, standardizing spacing around operations and parentheses, and canonicalizing formatting. We then compared the normalized versions to identify cases where the generated output did not change significantly from the original function. These functions were discarded to ensure that each retained sample reflected a true functional transformation corresponding to the target vulnerability. A summary of the filtering results, including the counts of removed multifunction lines, truncated samples, unchanged outputs, and final retained samples, is presented in Table I. The lowest retention was observed for CWE-20, at 58.32%, and the highest retention was found for CWE-787, at 64%. After filtering, although the usable samples are high, we had a significant drop in the number of samples.

## IV. Experiments and Results

### A. GNN Model

The GNN architecture (Fig. 2) is implemented with PyTorch Geometric [14]. It employs stacked GeneralConv layers [15] to produce node embeddings, configured with mean aggregation and dot-product attention. Each layer applies Parametric ReLU for adaptive nonlinearity and GraphNorm [16] for within-graph normalization.

A global mean pooling operation aggregates node embeddings into a graph-level vector, which is transformed by a linear layer into two dimensions for binary classification. Training uses class-weighted cross-entropy loss with the Adam optimizer, default hyperparameters, 50 epochs, and a batch size of 256. During inference, predictions are obtained via `argmax`, while softmax outputs provide probabilistic confidence scores, useful for uncertainty-aware applications.

We ran the experiments on a single A100 GPU (80GB) with dual Xeon Gold 6338 CPUs and 238 GB RAM, though training primarily leveraged the GPU.

### B. Input Preparation

To analyze the structural and semantic properties of the generated code, we converted all vulnerable and non-vulnerable functions into CPGs using the Joern framework [17]. Joern is a state-of-the-art open-source code analysis platform that parses source code and constructs an intermediate representation combining abstract syntax trees, control flow graphs, and data flow graphs into a unified graph format. The code was loaded into Joern's workspace, where it was automatically transformed into CPGs through a pipeline of parsing and graph construction steps. This representation enabled fine-grained program analysis by exposing relationships between variables, operations, and control structures. The resulting CPGs were exported in a serialized format suitable for downstream tasks such as machine learning-based vulnerability classification and graph-based feature extraction. Additionally, we used *StarCoder*'s [18] (a language model trained with multiple languages) tokenizer to generate embeddings, on nodes in the CPGs for the training process, which is inspired by other vulnerability detection ML approaches [7], [19].

### C. Training and Evaluation Metrics

To assess the performance of our GNN model, we employed standard evaluation metrics, including F1 score and accuracy. Let *TP, FP, FN, TN* represent true positives, false positives, false negatives, and true negatives, respectively. In our context:

- **True Positive (TP)**: A generated functioned labeled as vulnerable is correctly identified as vulnerable.
- **False Positive (FP)**: A non-vulnerable function is incorrectly labeled as vulnerable.
- **False Negative (FN)**: A vulnerable function is incorrectly labeled as non-vulnerable.
- **True Negative (TN)**: A non-vulnerable function is correctly labeled as non-vulnerable.

We define the evaluation metrics as follows:

- **Precision =** $\frac{TP}{TP+FP}$ measures the correctness of positive predictions.
- **Recall =** $\frac{TP}{TP+FN}$ measures the model's ability to identify all relevant positive cases.
- **F1 Score =** $2 \cdot \frac{\textbf{Precision} \cdot \textbf{Recall}}{\textbf{Precision} + \textbf{Recall}}$ provides the harmonic mean of precision and recall, balancing both metrics.
- **Accuracy =** $\frac{TP+TN}{TP+FP+FN+TN}$ reflects the overall proportion of correct predictions.

Precision and recall provide insight into the model's ability to correctly identify positive instances, critical for vulnerability detection tasks where False Positives (FPs) and False Negatives (FNs) carry different practical implications. Accuracy is a reliable indicator of model performance under balanced class distributions. Thus, in contrast to prior studies that contended with significant class imbalance, the nature of our dataset allows accuracy to be a more insightful evaluation metric. However, to provide a more comprehensive evaluation, especially considering potential nuances in false positives and false negatives, we also report the F1 score. The F1 score balances precision and recall, making it a valuable metric for assessing model performance even when some imbalance exists. These metrics collectively enable a comprehensive evaluation of the GNN's effectiveness and generalizability.

TABLE I
FILTERING OUTCOMES FOR LLM-GENERATED VULNERABILITY SAMPLES BY CWE TYPE

| CWE Type | Total Samples | Multi-funcs Removed | Truncated Removed | No-change Removed | Samples After Filtering | After CPG Conversion (Usable) | Retention (%) |
|---|---|---|---|---|---|---|---|
| CWE-787 | 6016 | 21 | 380 | 683 | 4932 | 3850 | 64.00 |
| CWE-125 | 6720 | 49 | 383 | 971 | 5317 | 4152 | 61.79 |
| CWE-119 | 6400 | 80 | 449 | 1123 | 4748 | 3760 | 58.75 |
| CWE-20 | 6696 | 60 | 423 | 1243 | 4970 | 3905 | 58.32 |
| CWE-416 | 6820 | 99 | 441 | 1148 | 5132 | 4117 | 60.37 |

TABLE II
TRAIN/TEST SET COMPOSITION BY CWE TYPE AND LLM VULNERABILITY PROPORTION. EACH CWE INCLUDES SYNTHETIC (LLM-GENERATED) AND REAL VULNERABILITY COUNTS.

| CWE Type | Source | 0% LLM-Vul | 50% LLM-Vul | 100% LLM-Vul | TestSet1 | TestSet2 |
|---|---|---|---|---|---|---|
| CWE-787 | Synthetic | 0 | 1,541 | 3,080 | 770 | 0 |
| | Real | 318 | 318 | 318 | 96 | 96 |
| CWE-125 | Synthetic | 0 | 1,661 | 3,322 | 830 | 0 |
| | Real | 538 | 538 | 538 | 80 | 80 |
| CWE-119 | Synthetic | 0 | 1,504 | 3,008 | 752 | 0 |
| | Real | 679 | 679 | 679 | 19 | 19 |
| CWE-20 | Synthetic | 0 | 1,562 | 3,124 | 781 | 0 |
| | Real | 442 | 442 | 442 | 11 | 11 |
| CWE-416 | Synthetic | 0 | 1,647 | 3,294 | 823 | 0 |
| | Real | 228 | 228 | 228 | 28 | 28 |
| Others | Synthetic | 0 | 0 | 0 | 0 | 0 |
| | Real | 2,350 | 2,350 | 2,350 | 359 | 359 |
| **Total Synthetic Vul** | | 0 | 7,914 | 15,828 | 3,956 | 0 |
| **Total Real Vul** | | 4,555 | 4,555 | 4,555 | 593 | 593 |
| **Total Real Nonvul** | | 159,856 | 159,856 | 159,856 | 23,355 | 23,355 |
| **Total Size** | | 164,411 | 172,325 | 180,239 | 27,904 | 23,948 |
| **TestSet1 Result** | Accuracy / F1 | 0.8080 / 0.0595 | 0.9780 / 0.9317 | 0.9786 / 0.9327 | – | – |
| **TestSet2 Result** | Accuracy / F1 | 0.9517 / 0.2126 | 0.9584 / 0.2133 | 0.9650 / 0.2380 | – | – |

### *D. Dataset and Statistics*

*ResourceVul* consists of function-level code samples into which vulnerabilities were injected using the DeepSeek-Coder-1.3b-Instruct model. Vulnerabilities were induced across multiple CWE types to ensure diversity in vulnerability classes. Table I summarizes the filtering outcomes applied to the generated functions. Samples were removed if they spanned multiple functions, had truncated outputs, or showed no syntactic changes after injection. Using the retained filtered samples, we assessed vulnerability detectability with two tools: Cppcheck and Flawfinder [3]. Detection rates varied by CWE type, but generally showed that:

- **Cppcheck** detected approximately 50% of the injected vulnerabilities.
- **Flawfinder** detected around 30%.

Table III summarizes the detection outcomes of LLM-generated vulnerabilities across different CWE types using Flawfinder and Cppcheck. The table presents detection rates after filtering and before conversion to CPGs, illustrating the variability in tool sensitivity and highlighting which vulnerability classes are more readily detected by each tool. Although DeepSeek-Coder effectively injected a substantial number of valid vulnerabilities, static analysis tools detected only a subset of them, highlighting inherent challenges in automated vulnerability detection. It is important to note that, despite their limitations, including FP, FN, and reliance on pattern-based heuristics, static analyzers remain valuable for their scalability, efficiency, and provision of objective, repeatable metrics that guide further validation efforts. Following vulnerability detection, the final set of usable samples was converted to CPGs for further program analysis. The retention rate, defined as the proportion of original generated samples retained after filtering and successful CPG conversion, varied between 58%

and 64% across CWE types, demonstrating the model's ability to produce syntactically valid and analyzable vulnerable code snippets.

### E. Model Performance

The ablation study demonstrated a positive correlation between the amount of LLM-Vul training data and generalization performance, measured by the F1-score and accuracy. As we incrementally increased the volume of synthetic vulnerability-injected data, the model's ability to correctly identify vulnerabilities in previously unseen code improved consistently. When increasing the proportion of LLM-Vul training data from 0% to 50%, the model's accuracy improved by 0.67%, while F1 increased by 0.07%. Increasing the proportion from 0% to 100%, the model's accuracy increased by 1.33% and F1 increased by 2.54%. This performance gain suggests that the prompt-injected CWE examples provide valuable training benefits when combined with real, vulnerable samples, enhancing the model's capacity to generalize to real-world vulnerabilities beyond those seen during training.

## V. Discussion

### A. Limitations

Despite iterative refinement, our approach cannot consistently identify optimal prompts across all vulnerability types and code contexts. While prompt engineering improved LLM outputs, results remained highly sensitive to phrasing and context. Prompts effective for one CWE often failed for another, revealing the instability of current prompting methods and the need for more controllable approaches.

A second limitation concerns CWE correctness. We generated 19,784 synthetic samples but did not systematically verify that each contained the requested CWE. Manual inspection of ten samples showed all were vulnerable, though often with different or multiple CWEs. Figs. 3 and 4 illustrate this: the first sample includes CWE-416 but also CWE-415 and CWE-590; the second, prompted for CWE-416, introduces CWE-401, CWE-476, and CWE-690. These cases underscore both the richness and labeling challenges of synthetic data. Since our model performs binary classification, these inconsistencies do not introduce training noise; however, they would for multi-class classification by CWE.

Our work is limited to binary classification (vulnerable vs. non-vulnerable). Extending it to multi-class CWE detection would require more reliable labeling. Experiments were also restricted to C/C++ functions; applying the method to other languages would need adaptation for syntax and vulnerability differences.

Automated validation revealed varying vulnerability quality. As shown in Table III, Flawfinder detected 25–57% of samples and Cppcheck 43–65%, with memory-safety CWEs (e.g., CWE-787, CWE-416) more consistently identified than semantic ones (e.g., CWE-20). This suggests that synthetic vulnerabilities partially reflect real patterns but exhibit domain shifts that limit generalization.

Scalability introduced practical challenges. The LLM occasionally added auxiliary code, explanations, or truncated outputs, requiring automated filtering and length-based exclusion. While these steps improved consistency, they highlight trade-offs between scale and fidelity in LLM-based data generation.

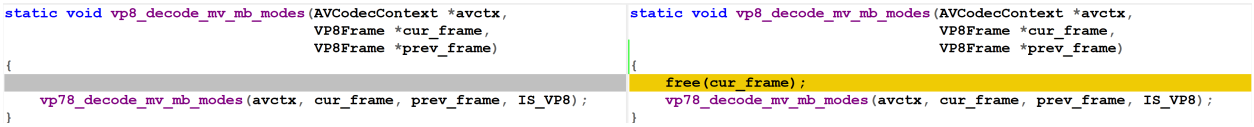
Finally, although synthetic data improved accuracy and modestly increased F1 scores, overall F1 remained low due to (i) severe class imbalance, fewer than 600 vulnerable versus 23,000 safe samples, and (ii) mismatches between synthetic and real vulnerabilities. Addressing these may require threshold tuning, curriculum-style pretraining, or loss functions for imbalance. Per-CWE analysis shows LLM augmentation benefits memory-safety categories most, confirming its value as augmentation rather than replacement for real data.

### B. Answers to Research Questions

**RQ1: Does training on LLM-injected vulnerabilities improve generalization to real-world code?:** Our ablation results (Table II) show consistent gains in accuracy (+1.3 percentage points) and F1-score (+2.5 points). These improvements are partly due to the balanced injection strategy: unlike PrimeVul, where vulnerable samples are severely underrepresented (6,968 vs. 228k benign), InfectPrompt ensures that each targeted CWE has thousands of paired examples. This directly mitigates class imbalance and enables fairer model training.

Moreover, CWE-specific analysis highlights that memory-safety flaws (CWE-787, CWE-416) are injected more reliably, with higher retention and tool detection rates, whereas semantic vulnerabilities (CWE-20) are less consistent. This asymmetry indicates that LLM augmentation is particularly beneficial for certain CWE families but not universally effective.

**RQ2: Can LLM-injected vulnerabilities serve as a substitute for real-world vulnerable samples in training?** Although models trained with synthetic vulnerabilities generalize better than those trained on real data alone, our analysis shows that synthetic data cannot fully replace real-world samples. Synthetic injections are effective for certain CWE types, particularly memory safety categories, but they lack the full diversity and subtlety of naturally occurring vulnerabilities. This leads to limited recall on real test cases, keeping F1 relatively

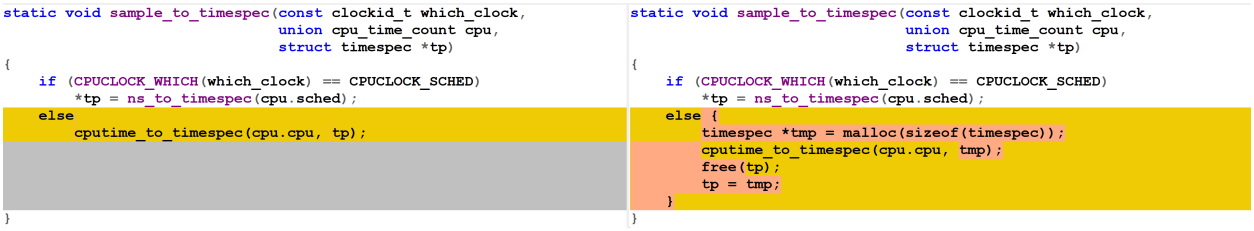


```
static void vp8_decode_mv_mb_modes(AVCodecContext *avctx,
                                   VP8Frame *cur_frame,
                                   VP8Frame *prev_frame)
{

    vp78_decode_mv_mb_modes(avctx, cur_frame, prev_frame, IS_VP8);
}
```

```
static void vp8_decode_mv_mb_modes(AVCodecContext *avctx,
                                   VP8Frame *cur_frame,
                                   VP8Frame *prev_frame)
{
    free(cur_frame);
    vp78_decode_mv_mb_modes(avctx, cur_frame, prev_frame, IS_VP8);
}
```

Fig. 3. Synthetic example which was prompted to induce a CWE-416 weakness, with correct CWE induced along with other potential weaknesses



```
static void sample_to_timespec(const clockid_t which_clock,
                               union cpu_time_count cpu,
                               struct timespec *tp)
{
    if (CPUCLOCK_WHICH(which_clock) == CPUCLOCK_SCHED)
        *tp = ns_to_timespec(cpu.sched);
    else
        cputime_to_timespec(cpu.cpu, tp);

}
```

```
static void sample_to_timespec(const clockid_t which_clock,
                               union cpu_time_count cpu,
                               struct timespec *tp)
{
    if (CPUCLOCK_WHICH(which_clock) == CPUCLOCK_SCHED)
        *tp = ns_to_timespec(cpu.sched);
    else {
        timespec *tmp = malloc(sizeof(timespec));
        cputime_to_timespec(cpu.cpu, tmp);
        free(tp);
        tp = tmp;
    }
}
```

Fig. 4. Synthetic example which was prompted to induce a CWE-416 weakness, and incorrect CWEs induced

TABLE III
DETECTION RATES OF LLM-INJECTED VULNERABILITIES BY FLAWFINDER AND CPPCHECK ACROSS CWE TYPES

| **Tool** | **CWE-787** | **CWE-125** | **CWE-119** | **CWE-20** | **CWE-416** |
|---|---|---|---|---|---|
| Flawfinder (all filtered samples) | 50.51% (2491/4932) | 29.33% (1559/5317) | 56.76% (2695/4748) | 25.21% (1253/4969) | 27.89% (1431/5133) |
| Cppcheck (40 samples per CWE) | 43% (17/40) | 50% (20/40) | 50% (20/40) | 45% (18/40) | 65% (26/40) |

low. Thus, LLM-generated data is best positioned as an augmentation strategy rather than a standalone substitute.

**RQ3: How does the success rate of LLM-injected vulnerabilities vary across different CWE types?** Our filtering analysis (Table I) and static analysis validation (Table III) reveal variation in both generation quality and detection rates. For instance, CWE-787 retained the highest proportion of usable samples (64%) and achieved strong detection rates with Flawfinder (50.5%) and Cppcheck (43%). In contrast, CWE-20 showed lower retention (58.3%) and weaker tool detection rates (25.2%). This suggests that LLMs are more effective at generating structural memory safety flaws than semantic or logic-based errors. As a result, synthetic augmentation is unevenly beneficial across CWEs, with memory safety classes showing the greatest promise.

## VI. CONCLUSION AND FUTURE WORK

This work demonstrates that while LLMs exhibit promising capabilities in injecting realistic vulnerabilities into source code, their outputs are not yet sufficient to fully replace real-world vulnerable samples, especially when evaluating ML models on real, unseen code. The nuances of real-world vulnerabilities remain challenging for LLMs to replicate with complete fidelity. However, our proposed approach, *InfectPrompt*, offers a scalable and systematic method for generating diverse and syntactically valid vulnerability-injected code samples. This has significant implications for augmenting training datasets in ways that promote better coverage across a wider range of CWEs. By improving the diversity and balance of synthetic vulnerability data, InfectPrompt contributes to the broader goal of enhancing the robustness and generalizability of automated vulnerability detection systems.

Several avenues remain for future exploration. First, the use of commercially available LLMs with enhanced capabilities beyond those of free-tier versions presents an opportunity to improve the accuracy and realism of vulnerability injection. Leveraging these advanced models may yield more diverse and semantically coherent examples. Second, fine-tuning LLMs specifically on curated datasets of vulnerability descriptions could enhance their coverage of CWE classes and help mitigate hallucinations during code generation. Finally, further investigation is needed into the distributional effects of LLM-generated synthetic data on downstream model performance. In particular, evaluating how these data impact learning in various vulnerability categories will be essential to understand their efficacy and limitations.


## ACKNOWLEDGMENT

This research is supported in part by the National Science Foundation under Award No. CNS-2243619. The opinions, findings, and conclusions or recommendations expressed in the article are those of the authors and do not necessarily reflect the views of the National Science Foundation.

We also acknowledge and thank Miles Farmer from the University of Missouri-Columbia for helping us set up and use the GNN model for vulnerability detection.

## REFERENCES


[1] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks," Sep. 2019, arXiv:1909.03496 [cs]. [Online]. Available: http://arxiv.org/abs/1909.03496

[2] J. He and M. Vechev, "Large Language Models for Code: Security Hardening and Adversarial Testing," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. Copenhagen Denmark: ACM, Nov. 2023, pp. 1865–1879. [Online]. Available: https://dl.acm.org/doi/10.1145/3576915.3623175

[3] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner, "DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection," in *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. Hong Kong China: ACM, Oct. 2023, pp. 654–668. [Online]. Available: https://dl.acm.org/doi/10.1145/3607199.3607242

[4] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories*. Seoul Republic of Korea: ACM, Jun. 2020, pp. 508–512. [Online]. Available: https://dl.acm.org/doi/10.1145/3379597.3387501

[5] M. Shahzad, J. Wilson, I. Al Azher, H. Alhoori, and M. Rahimi, "From Theory to Practice: Code Generation Using LLMs for CAPEC and CWE Frameworks," in *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*. Ottawa, ON, Canada: IEEE, May 2025, pp. 137–144. [Online]. Available: https://ieeexplore.ieee.org/document/11028365/

[6] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability Detection with Code Language Models: How Far Are We?" Jul. 2024, arXiv:2403.18624 [cs]. [Online]. Available: http://arxiv.org/abs/2403.18624

[7] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "VulCNN: an image-inspired scalable vulnerability detection system," in *Proceedings of the 44th International Conference on Software Engineering*. Pittsburgh Pennsylvania: ACM, May 2022, pp. 2365–2376. [Online]. Available: https://dl.acm.org/doi/10.1145/3510003.3510229

[8] G. Nikitopoulos, K. Dritsa, P. Louridas, and D. Mitropoulos, "CrossVul: a cross-language vulnerability dataset with commit data," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Athens Greece: ACM, Aug. 2021, pp. 1565–1569. [Online]. Available: https://dl.acm.org/doi/10.1145/3468264.3473122

[9] G. Bhandari, A. Naseer, and L. Moonen, "CVEfixes: automated collection of vulnerabilities and their fixes from open-source software," in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*. Athens Greece: ACM, Aug. 2021, pp. 30–39. [Online]. Available: https://dl.acm.org/doi/10.1145/3475960.3475985

[10] M. B. U. Ahmed, N. S. Harzevili, J. Shin, H. V. Pham, and S. Wang, "Secvuleval: Benchmarking llms for real-world c/c++ vulnerability detection," *arXiv preprint arXiv:2505.19828*, 2025.

[11] T. Dias, J. Gonçalves, E. Maia, and I. Praça, "Large Language Models for Source Code Vulnerability Detection: A DiverseVul Analysis," Jul. 2024. [Online]. Available: https://www.researchsquare.com/article/rs-4583707/v1

[12] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities," Oct. 2024, arXiv:2311.16169 [cs]. [Online]. Available: http://arxiv.org/abs/2311.16169

[13] C. Arora, A. I. Sayeed, S. Licorish, F. Wang, and C. Treude, "Optimizing Large Language Model Hyperparameters for Code Generation," Aug. 2024, arXiv:2408.10577 [cs]. [Online]. Available: http://arxiv.org/abs/2408.10577

[14] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[15] J. You, Z. Ying, and J. Leskovec, "Design space for graph neural networks," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33, 2020, pp. 17 009–17 021.

[16] T. Cai, S. Luo, K. Xu, D. He, T.-Y. Liu, and L. Wang, "GraphNorm: A principled approach to accelerating graph neural network training," in *Proc. 38th Int. Conf. on Machine Learning*, vol. 139, 2021, pp. 1204–1215.

[17] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and Discovering Vulnerabilities with Code Property Graphs," in *2014 IEEE Symposium on Security and Privacy*. San Jose, CA: IEEE, May 2014, pp. 590–604. [Online]. Available: http://ieeexplore.ieee.org/document/6956589/

[18] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "Starcoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.

[19] A. Watson, E. Ufuktepe, and K. Palaniappan, "Detecting software code vulnerabilities using 2d convolutional neural networks with program slicing feature maps," in *2022 IEEE Applied Imagery Pattern Recognition Workshop (AIPR)*. IEEE, 2022, pp. 1–9.

[20] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and Discovering Vulnerabilities with Code Property Graphs," in *2014 IEEE Symposium on Security and Privacy*. San Jose, CA: IEEE, May 2014, pp. 590–604. [Online]. Available: http://ieeexplore.ieee.org/document/6956589/

[21] M. Beller, R. Bholanath, S. McIntosh, and A. Zaidman, "Analyzing the State of Static Analysis: A Large-Scale Evaluation in Open Source Software," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. Suita: IEEE, Mar. 2016, pp. 470–481. [Online]. Available: http://ieeexplore.ieee.org/document/7476667/

[22] W. Charoenwet, P. Thongtanunam, V.-T. Pham, and C. Treude, "An Empirical Study of Static Analysis Tools for Secure Code Review," Jul. 2024, arXiv:2407.12241 [cs]. [Online]. Available: http://arxiv.org/abs/2407.12241

[23] L. Salewski, S. Alaniz, I. Rio-Torto, E. Schulz, and Z. Akata, "In-Context Impersonation Reveals Large Language Models' Strengths and Biases."

[24] A. Mohsin, H. Janicke, A. Wood, I. H. Sarker, L. Maglaras, and N. Janjua, "Can We Trust Large Language Models Generated Code? A Framework for In-Context Learning, Security Patterns, and Code Evaluations Across Diverse LLMs," Jun. 2024, arXiv:2406.12513 [cs]. [Online]. Available: http://arxiv.org/abs/2406.12513

[25] D. Fernandes, J. P. Matos-Carvalho, C. M. Fernandes, and N. Fachada, "DeepSeek-V3, GPT-4, Phi-4, and LLaMA-3.3 generate correct code for LoRaWAN-related engineering tasks," *Electronics*, vol. 14, no. 7, p. 1428, Apr. 2025, arXiv:2502.14926 [cs]. [Online]. Available: http://arxiv.org/abs/2502.14926

[26] W. Hou and Z. Ji, "Comparing Large Language Models and Human Programmers for Generating Programming Code," *Advanced Science*, vol. 12, no. 8, p. 2412279, Feb. 2025. [Online]. Available: https://advanced.onlinelibrary.wiley.com/doi/10.1002/advs.202412279

[27] ——, "Comparing Large Language Models and Human Programmers for Generating Programming Code," *Advanced Science*, vol. 12, no. 8, p. 2412279, Feb. 2025. [Online]. Available: https://advanced.onlinelibrary.wiley.com/doi/10.1002/advs.202412279

[28] M. L. Siddiq, J. C. S. Santos, S. Devareddy, and A. Muller, "SALLM: Security Assessment of Generated Code," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops*, Oct. 2024, pp. 54–65, arXiv:2311.00889 [cs]. [Online]. Available: http://arxiv.org/abs/2311.00889

[29] B. Zeng, Q. Zhang, C. Zhou, G. Go, Y. Jiang, and H. Shi, "Inducing Vulnerable Code Generation in LLM

Coding Assistants," Apr. 2025, arXiv:2504.15867 [cs]. [Online]. Available: http://arxiv.org/abs/2504.15867

[30] A. Garg, R. Degiovanni, M. Papadakis, and Y. L. Traon, "On the Coupling between Vulnerabilities and LLM-Generated Mutants: A Study on Vul4J Dataset," in *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. Toronto, ON, Canada: IEEE, May 2024, pp. 305–316. [Online]. Available: https://ieeexplore.ieee.org/document/10638554/